

Math Foundations of Machine Learning

When Model Meets Data

Gabriel Wendell Celestino Rocha

July 2025

Table of Contents

Data, Models, and Learning

- Data as Vectors

- Models as Functions

- Models as Probability Distributions

- Learning is Finding Parameters

Empirical Risk Minimization

- Hypothesis Class of Functions

- Loss Function for Training

- Regularization to Reduce Overfitting

- Cross-Validation

Parameter Estimation

- Maximum Likelihood Estimation

- Maximum A Posteriori Estimation

- Model Fitting and Model Classes

Probabilistic Modeling and Inference



Data, Models, and Learning

Representing Data Numerically

Machine Learning (ML) models require input in **numerical format**.

1. Data typically comes in **tabular form**:
 - Rows ($\mathbf{x}_n$) $\implies$ instances/examples
 - Columns ($d = 1, \dots, D$) $\implies$ features/attributes
2. Not all data is naturally tabular:
 - Examples: genome sequences, social graphs, images, text, etc
3. **Goal**: Represent and manipulate data as **vectors** to apply mathematical techniques.
4. **Challenge**: Extract **meaningful features** often requires:
 - Domain knowledge
 - Feature engineering
 - Data science techniques

Tabular Format and Feature Engineering

Problem: Even tabular data often requires preprocessing.

1. Example: Categorical variables like Gender
 - Encode as: Male = 0, Female = 1 or Male = -1 and Female = +1
2. Domain-dependent encodings:
 - Degree $\implies$ ordinal scale (Bachelor < Master < PhD)
 - Postcode $\implies$ convert to latitude & longitude
3. Even “numerical” data may need **interpretation**.
4. Scale all features to **zero mean** and **unit variance**

$$\text{Standardized Feature} = \frac{x - \mu}{\sigma} .$$

Example

Name	Gender	Degree	Postcode	Age	Annual Salary
Aditya	M	MSc	W21BG	36	89,563
Bob	M	PhD	EC1A1BA	47	123,543
Chloé	F	BEcon	SW1A1BH	26	23,989
Daisuke	M	BSc	SE207AT	68	138,769
Elisabeth	F	MBA	SE10AA	33	113,888

Gender ID	Degree	Latitude	Longitude	Age	Annual Salary
-1	2	51.5073	0.1290	36	89.563
-1	3	51.5074	0.1275	47	123.543
+1	1	51.5074	0.1278	26	23.989
-1	1	51.5075	0.1281	68	138.769
+1	2	51.5074	0.1278	33	113.888

Table: Latitude and Longitude are in degrees and the Annual Salary is in thousands.

Vectors and Notation

In this course we adopt the following notation:

1. **Each row:** $\mathbf{x}_n \in \mathbb{R}^D$ (feature vector for example n)
2. **Total dataset:** $\mathbf{X} \in \mathbb{R}^{N \times D}$
3. **Indexing factors:**
 - $n = 1, \dots, N \implies$ Number of examples
 - $d = 1, \dots, D \implies$ Number of features

Convention

- **Row vectors:** from database community
- **Column vectors:** preferred in ML math

Supervised Learning Example

Task: Predict Annual Salary from Age

- **Inputs:** $\mathbf{x}_n \in \mathbb{R} = \text{Age}$
- **Outputs:** $y_n \in \mathbb{R} = \text{Salary}$
- **Dataset:**
 $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$

We are interested in the salary of a person aged sixty ($x = 60$), which is not part of the training data.

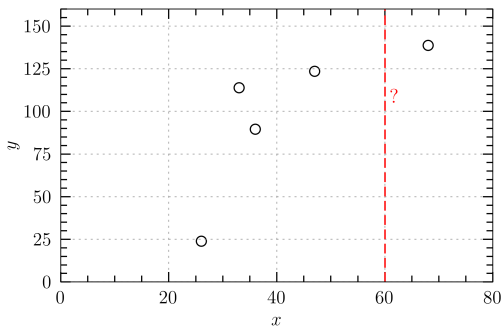


Figure: Toy data for linear regression.

Mathematical Tools

Representing data as vectors enables:

1. Use of linear algebra
2. Distance/similarity computations
 - Euclidean distance
 - Cosine similarity
 - etc
3. Geometry of data
4. Optimization over vector spaces

Strategies for data representation:

1. **Dimensionality Reduction**
 - PCA method
 - Uses **eigenvalues/SVD**
2. **Nonlinear High-Dimensional Maps**
 - Feature maps $\phi(\mathbf{x}_n)$
 - **Kernel methods** and **SVMs**

Models as Functions

Definition: Predictor Function

A **predictor** is a function:

$$f : \mathbb{R}^D \rightarrow \mathbb{R} .$$

- **Input:** A feature vector $\mathbf{x} \in \mathbb{R}^D$
- **Output:** A real-valued prediction $f(\mathbf{x}) \in \mathbb{R}$

- Typically, the input is a row in the dataset, and the output is the predicted label or value.
- The output here is scalar (e.g., salary, price, temperature, etc).

In theory, f could be **any function**, but this would require **functional analysis**. Instead, we restrict to **linear functions**.

Linear Predictor

Definition: Linear Predictor

The linear model for a predictor is given by:

$$f(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x} + \theta_0, \quad (1)$$

- $\boldsymbol{\theta} \in \mathbb{R}^D$ is a weight vector;
- $\theta_0 \in \mathbb{R}$ is a bias (intercept) term.

This form captures only linear relationships and is the foundation of **linear regression**, **logistic regression**, etc.

Why Linear Models

- Mathematical tractability
- Easily interpretable
- Compatible with convex optimization
- Foundation for many advanced ML models

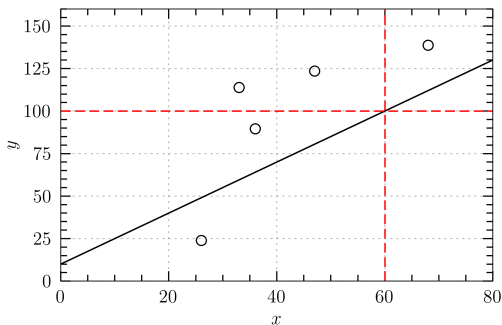


Figure: Example function (black solid diagonal line) and its prediction at $x = 60$, i.e., $f(60) = 100$.

Models as Probability Distributions

We often consider our observations to be **noisy** (i.e., observations = signal + noise). Because of this, we expect that by applying ML models we will be able to separate the signal from the noise.

Because of this, it is desirable that we are able to make predictions with a certain level of **uncertainty** that can be quantified in some way.

- **Deterministic Model:**
Outputs a single value $\implies$ gives no information about confidence or uncertainty
- **Probabilistic Model:**
Outputs a **distribution** over predictions $\implies$ allows expressing *confidence, variability, and risk*

From Functions to Distributions over Functions

Instead of a fixed function f , define a **distribution over possible functions**.

- Predictive output:

$$f(\mathbf{x}) \sim \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x}))$$

- **Result:** Predictions are **Gaussian-distributed** (or other distributions)

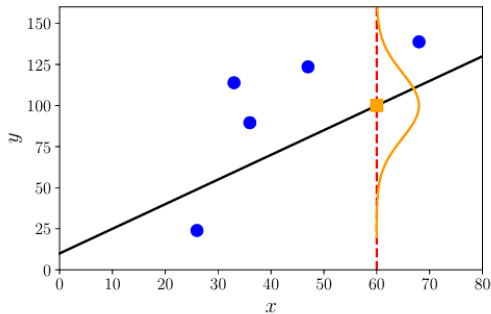


Figure: Example function (black solid diagonal line) and its predictive uncertainty at $x = 60$ (drawn as a Gaussian).

Learning is Finding Parameters

“The goal of learning is to find a model and its parameters such that the resulting predictor performs well on unseen data.”

The three phases of Machine Learning:

1. **Prediction (Inference)**

Apply a trained model to **unseen test data**. The output is $\hat{y} = f(x)$.

2. **Training (Parameter Estimation)**

Fit the model using **training data**

3. **Model Selection (Hyperparameter Tuning)**

Choose the best **model structure** and **hyperparameters**

On the terminology of “inference”

- **Non-probabilistic ML:** Making a prediction
- **Probabilistic ML:** Estimating parameters of distributions

Principles of Learning

Training Phase: Adjust model parameters using training data

1. Point Estimation (most common approach)

- Estimate best single value for parameters
- Used in both probabilistic and non-probabilist models: uses *Empirical Risk Minimization* (ERM)

2. Bayesian Inference

- Infer a distribution over parameters
- Requires **probabilistic models**: use *Maximum Likelihood Estimation* (MLE)

How we optimize the parameters?

- Parameters are found by **optimization** using methods like de gradient descent, hill-climbing algorithms, and Newton-types methods.
- We use the *loss function* or *likelihood function* as the **objective**.

Fitting vs Generalization

To perform well, a model must:

1. **Fit** the training data
2. **Generalize** to unseen data

How do we simulate generalization?

1. Use **cross-validation**
2. Apply **regularization**
 - Add penalty to complexity, e.g., $\lambda \|\theta\|^2$

Abduction from a Philosophical View

“Abduction *is inference to the best explanation.*
It is not deduction or induction.”

In ML, choosing the best model from among many is **abduction**. For example, selecting a regularized predictor that balances accuracy and simplicity.

Hyperparameters and Model Selection

Hyperparameters

- Not directly trained from data
- Control model structure

Model Selection

- Choose best model based on **validation performance**
- Often done using **Nested Cross-Validation**

Hyperparameters are tuned **around** model training

Type	Description	Example
Parameter	Trained by optimization	Weights θ , bias
Hyperparameters	Set manually or via grid search	Learning rate, model depth

- Parameters = explicit in likelihood
- Hyperparameters = control prior/distribution over parameters



Empirical Risk Minimization

Empirical Risk Minimization

The central principle guideline of supervised machine learning is:

“Learn a function *from data by minimizing a measure of* **error** (i.e., a **loss**) *between prediction and true outputs.”*

We break it into five components:

1. Hypothesis class of functions
2. Loss function
3. Optimization procedure
4. Regularization
5. Cross-Validation

Hypothesis Class of Functions

Problem Setup

1. **Given** N examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, with $\mathbf{x}_n \in \mathbb{R}^D, y_n \in \mathbb{R}$,
2. **Learn** a function $f(\cdot; \boldsymbol{\theta})$ that maps inputs to outputs $f: \mathbb{R}^D \rightarrow \mathbb{R}$,
3. **Parametrized** by $\boldsymbol{\theta} \in \mathbb{R}^{D+1}$.

- We aim to **find**:

$$f(\mathbf{x}_n, \boldsymbol{\theta}^*) \approx y_n \quad , \quad \forall n = 1, \dots, N$$

Here, we use the notation $\hat{y}_n = f(\mathbf{x}_n, \boldsymbol{\theta}^*)$ to represent the output of the predictor. We also restrict our hypothesis class to **affine functions**, i.e.,

$$f(\mathbf{x}_n, \boldsymbol{\theta}^*) = \boldsymbol{\theta}^T \tilde{\mathbf{x}}_n = \theta_0 + \sum_{d=1}^D \theta_d x_n^{(d)} \quad , \quad \text{where: } \begin{cases} \tilde{\mathbf{x}}_n = [1, x_n^{(1)}, x_n^{(2)}, \dots, x_n^{(D)}]^T & \in \mathbb{R}^{D+1} \\ \boldsymbol{\theta} = [\theta_1, \theta_2, \dots, \theta_D]^T & \in \mathbb{R}^D \end{cases} .$$

Loss Function for Training

We now define how to measure prediction error using a *loss function* $\ell(y_n, \hat{y}_n)$.

Loss Function

Given a true label y_n and a prediction $\hat{y}_n = f(\mathbf{x}_n, \boldsymbol{\theta}^*)$, we define a **non-negative loss function**:

$$\ell(y_n, \hat{y}_n) : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0} .$$

The *empirical risk* is the average loss over training data:

$$\mathbf{R}_{\text{emp}}(f, \mathbf{X}, \mathbf{y}) = \frac{1}{N} \sum_{n=1}^N \ell(y_n, f(\mathbf{x}_n, \boldsymbol{\theta}^*)) = \frac{1}{N} \sum_{n=1}^N \ell(y_n, \hat{y}_n) .$$

Main Assumption: The data is i.i.d (*independent and identically distributed*), so we can treat the empirical risk as a good proxy for generalization.

- $\mathbf{X} \in \mathbb{R}^{N \times D}$: Matrix of inputs vectors
- $\mathbf{y} \in \mathbb{R}^N$: Vector of labels

Least-Square Loss

Consider the *squared loss* $\ell(y_n, \hat{y}_n) = (y_n - \hat{y}_n)^2$. Apply the ERM to minimize the empirical risk of this loss function.

This is known as the **least-squares problem**. There exists a closed-form analytic solution for this by solving the normal equations.

Beyond Empirical Risk

We don't just want low training error — we want low **expected risk** (also known as *population risk*):

$$\mathbf{R}_{\text{true}}(f) = \mathbb{E}_{(\mathbf{x}, y)} [\ell(y, f(\mathbf{x}))] .$$

The notation $\mathbf{R}_{\text{true}}(f)$ indicates that this is the true risk if we had access to an infinite amount of data. The expectation is over the (infinite) set of all possible data and labels.

Challenges:

1. How should we change our training procedure to generalize well?
2. How do we estimate expected risk from (finite) data?

Regularization to Reduce Overfitting

The Problem: Overfitting

Empirical Risk Minimization can lead to **overfitting** $\implies$ Training error is low, but test error is high.

$$\mathbf{R}_{\text{emp}} \left(f, \mathbf{X}^{\text{train}}, \mathbf{y}^{\text{train}} \right) \ll \mathbf{R}_{\text{emp}} \left(f, \mathbf{X}^{\text{test}}, \mathbf{y}^{\text{test}} \right) .$$

The Solution: Regularization

Add a **penalty term** to discourage complexity:

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \lambda \|\boldsymbol{\theta}\|^2 .$$

The second term $\lambda \|\boldsymbol{\theta}\|^2$:

- Encourages **smaller parameter norms**
- Helps prevent overfitting
- It's called the **regularizer**
- $\lambda > 0$ is the **regularization parameter**

Cross-Validation

Since we don't have access to the **true risk** $\mathbf{R}_{\text{true}}(f) = \mathbb{E}_{(\mathbf{x}, y)} [\ell(y, f(\mathbf{x}))]$ we estimate it using held-out data.

Cross-Validations (CV) Procedure

1. Split the dataset into K folds:
 - For each fold $k \in \{1, \dots, K\}$:
 - Train on $K - 1$ folds $\implies \mathcal{R}^{(k)}$
 - Validate on 1 fold $\implies \mathcal{V}^{(k)}$

2. Evaluate:

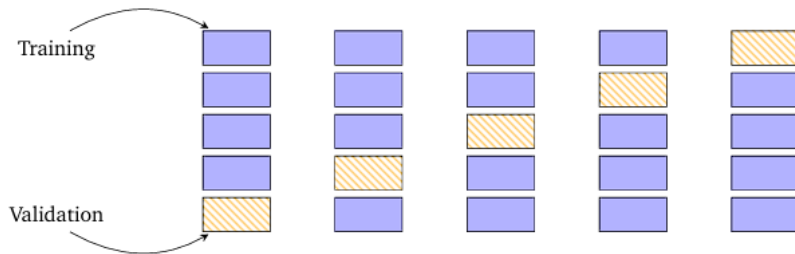
$$\mathbb{E}_{\mathcal{V}} [\mathcal{R}(f, \mathcal{V})] \approx \frac{1}{K} \sum_{k=1}^K \mathcal{R}(f^{(k)}, \mathcal{V}^{(k)}) ,$$

where $f^{(k)}$ is trained on the training set $\mathcal{R}^{(k)}$ and validated on the validation set $\mathcal{V}$.

Trade-Offs and Properties

The main properties of cross-validation are...

- Small validation sets $\Rightarrow$ High variance
- Large validation sets $\Rightarrow$ Less training data
- Cross-validation balances both
- Can be **embarrassingly parallel** (computationally scalable)



Alternatives to Cross-Validation: **Bootstrap**, **Jack-knife**, etc

Further Reading

Historical and Theoretical Foundations

- ERM originates from Vapnik's work (Statistical Learning Theory)
- Follow-up developments: *Hastie et al.* (2001), *Shalev-Schwartz & Ben-David* (2014)

Name	Description
Tikhonov	Classic ℓ^2 regularization (Ridge)
Ivanov	Regularization as constrained optimization
Bayesian	Penalty interpreted as prior probability

Philosophical Note

ERM is **distribution-free**, but assumes data arises from an **underlying probability law** $p(x,y)$. Unlike classical statistics, we do **not** specify this distribution.



Parameter Estimation

Maximum Likelihood Estimation

Main question

- *How can we use probability distributions to model our uncertainty due to the observation process and our uncertainty in the parameters of our predictors?*

Maximum Likelihood Estimation (MLE)

Core Idea: Estimate parameters θ that maximize the probability of observed data, i.e.,

$$\mathcal{L}_{\theta}(\theta) = -\log p(x|\theta) .$$

- $p(x|\theta)$ is the likelihood of data x given model parameters θ
- $\mathcal{L}_{\theta}$ is a function of θ (data is fixed)

This is the *negative log-likelihood*, which is minimized during training.

Supervised MLE with Gaussian Likelihood

Assumptions

- Observations: $(\mathbf{x}_n, y_n)$ with $\mathbf{x}_n \in \mathbb{R}^D$ and $y_n \in \mathbb{R}$
- Linear model: $\mathbb{E}[y_n | \mathbf{x}_n, \boldsymbol{\theta}] = \mathbf{x}_n^T \boldsymbol{\theta}$
- Gaussian noise: $\varepsilon_n \sim \mathcal{N}(0, \sigma^2)$

Likelihood

$$p(y_n | \mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{N}(y_n | \mathbf{x}_n^T \boldsymbol{\theta}, \sigma^2) .$$

Joint likelihood (i.i.d.)

$$p(\mathcal{Y} | \mathcal{X}, \boldsymbol{\theta}) = \prod_{n=1}^N p(y_n | \mathbf{x}_n, \boldsymbol{\theta}) .$$

Negative log-likelihood

$$\mathcal{L}(\boldsymbol{\theta}) = - \sum_{n=1}^N \log p(y_n | \mathbf{x}_n, \boldsymbol{\theta}) .$$

Gaussian case

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2\sigma^2} \sum_{n=1}^N \left(y_n - \mathbf{x}_n^T \boldsymbol{\theta} \right)^2 - \sum_{n=1}^N \log \frac{1}{\sqrt{2\pi\sigma^2}} .$$

This is equivalent to **least-square regression**.

Maximum A Posteriori Estimation

Maximum A Posteriori Estimation (MAP)

Core Idea: Estimate model parameters θ given both the data and prior knowledge about the parameters.

We maximize the posterior:

$$\begin{aligned}\theta_{\text{MAP}} &= \arg \max_{\theta} [p(\theta | \mathbf{x})] = \arg \max_{\theta} [p(\mathbf{x} | \theta) \cdot p(\theta)] \\ &= \arg \min_{\theta} \{-\log [p(\mathbf{x} | \theta)] - \log [p(\theta)]\}\end{aligned}$$

This is the **negative log-posterior**, i.e.

$$\mathcal{L}_{\text{MAP}}(\theta) = \mathcal{L}_{\text{MLE}}(\theta) + \mathcal{R}(\theta)$$

Use MAP when **prior information is available**. Unlike full Bayesian inference, MAP does not quantify uncertainty over θ .

MLE vs MAP

- MLE: $\theta_{\text{MLE}} = \arg \max_{\theta} [p(\mathbf{x}|\theta)]$
- MAP: $\theta_{\text{MAP}} = \arg \min_{\theta} [p(\theta|\mathbf{x})]$

Using Bayes' theorem:

$$p(\theta|\mathbf{x}) \propto p(\mathbf{x}|\theta) p(\theta) .$$

The objective function is therefore:

$$\theta_{\text{MAP}} = \arg \max_{\theta} \{ \log [p(\mathbf{x}|\theta)] \} + \log [p(\theta)] .$$

The **log-prior** $\log p(\theta)$ act as a **regularization term**. Using a Gaussian prior $\theta \sim \mathcal{N}(0, \Sigma)$ leads to:

$$\mathcal{L}_{\text{MAP}}(\theta) = \mathcal{L}_{\text{MLE}}(\theta) + \frac{1}{2} \theta^T \Sigma^{-1} \theta .$$

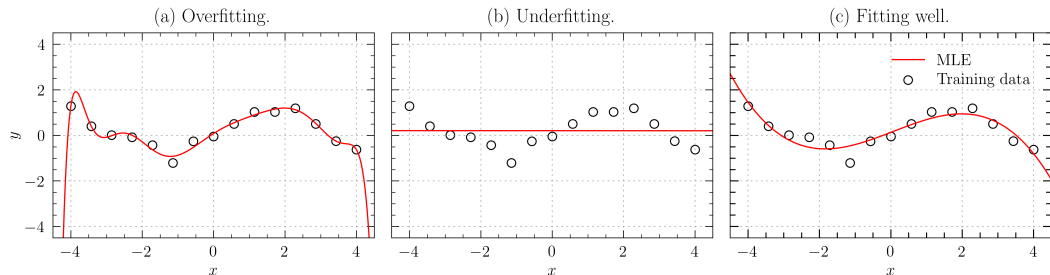
Model Fitting and Model Classes

Key Concepts

- We optimize parameters θ to fit a model class $\mathcal{M}_\theta$.
- Fit quality judged by training objectives (e.g., negative log-likelihood).

Three cases after optimization

1. Overfitting: $\mathcal{M}_\theta$ too rich
2. Underfitting: $\mathcal{M}_\theta$ too simple
3. Fitting well: $\mathcal{M}_\theta \approx \mathcal{M}^*$



Topics to Explore

1. Generalized Linear Models (GLMs)

$$p(y_n | \mathbf{x}_n, \boldsymbol{\theta}) = \varphi(\boldsymbol{\theta}^T \mathbf{x}_n) ,$$

where φ is a *link function*.

2. Historical Notes

- MLE proposed by Fisher in the 1930s
- Regularization $\approx$ Prior in Bayesian view

3. Debates

- Bayesian vs Frequentist
- MAP $\approx$ bridge between MLE and Bayesian inference



Probabilistic Modeling and Inference

Introduction to Probabilistic Modeling

1. **Objective:** Understand the role of probabilistic modeling in ML
2. **Key Point:** ML often requires modeling the generative process of data.
3. **Motivation:** Enables prediction, uncertainty quantification, and principled decision-making.

Coin Flip Example

Let $\mu \in [0, 1]$ be the probability of heads. The observed outcome $x \in \{\text{head}, \text{tail}\} \sim \text{Ber}(\mu)$. We model the **outcome** x **given** μ , but μ is **unknown**.

Goal: Learn about μ from observed outcomes $\{\mathbf{x}_n\}_{n=1}^N$.

Probabilistic Models

Definition: Models representing uncertainty via probabilistic distributions.

Central Object: The *joint distribution* $p(\mathbf{x}, \boldsymbol{\theta})$ over data and parameters

Encapsulates:

1. Prior $p(\boldsymbol{\theta})$
2. Likelihood $p(\mathbf{x}|\boldsymbol{\theta})$
3. Posterior $p(\boldsymbol{\theta}|\mathbf{x})$
4. Marginal Likelihood $p(\mathbf{x}) = \int p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta}) d\boldsymbol{\theta}$

Bayesian Inference

The goal of Bayesian inference is to infer $\boldsymbol{\theta}$ given $\mathbf{x}$ using a posterior distribution.

Bayes' Theorem

$$p(\boldsymbol{\theta}|\mathcal{X}) = \frac{p(\mathcal{X}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{X})}, \quad \text{where: } p(\mathcal{X}) = \int p(\mathcal{X}|\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta}.$$

Predictive Distribution

$$p(\mathbf{x}) = \int p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta} = \mathbb{E}_{p(\boldsymbol{\theta})}[p(\mathbf{x}|\boldsymbol{\theta})].$$

Average prediction over parameter uncertainty.

Point Estimation vs Bayesian Inference

Point Estimation vs Bayesian Inference

Aspect	Point Estimation (MLE/MAP)	Bayesian Inference
Output	θ^* (single point)	$p(\theta)$ (distribution)
Optimization	Yes	Yes (integration-based)
Prediction	$p(\mathbf{x} \theta^*)$	$\int p(\mathbf{x} \theta)p(\theta) d\theta$
Advantage	Simpler, efficient	Uncertainty-aware, robust
Challenge	Ignores uncertainty	Requires integration

Approximations in Bayesian Inference

Challenge: Intractable integrals in $p(\boldsymbol{\theta}|\mathcal{X}), p(\mathbf{x})$

Stochastic Approaches: MCMC (Metropolis-Hastings, HMC, etc)

Deterministic Approaches:

- Laplace Approximation
- Variational Inference
- Expectation Propagation

Applications:

- Large-scale Recommendation
- Reinforcement Learning
- Click-through Rate Prediction

Questions?